

DeepTriangle: An Actuarial Guide to Kevin Kuo's Deep Learning Reserving Model

Kevin Kuo's **DeepTriangle** is best understood as an attempt to modernise aggregate loss reserving without abandoning the development triangle itself.

Traditional chain ladder asks a fairly constrained question: given how claims have developed at each age in the past, what development factor should we apply to claims currently at that age?

DeepTriangle asks a broader question:

> Given the **whole development history observed so far**, together with patterns learned across many insurers and information about which insurer produced the triangle, what sequence of payments and case reserves should come next?

It is therefore still an **aggregate reserving model**. It does not model individual claims. Its observations are accident-year/development-year cells from development triangles. What changes is the way the development process is learned.

Paper at a glance

Question	DeepTriangle
---	---
Dataset	Real insurance regulatory data from National Association of Insurance Commissioners Schedule P
Granularity	Aggregate accident-year × development-year triangle cells
Coverage	Accident years 1988–1997, each ultimately developing to 10 years
Companies	50 insurers in each of four lines of business
Dataset modality	Sequential numerical triangle data plus categorical insurer identity
Inputs	Incremental paid loss, case outstanding, net earned premium and insurer identifier
Code available	Yes
Language	R, using Keras and TensorFlow
Main technique	Gated recurrent unit encoder-decoder
Other techniques	Sequence-to-sequence forecasting, multi-task learning, embeddings, dropout and ensembling
Learning paradigm	Supervised learning
Task	Multivariate regression and direct multi-step forecasting
Main output	Future incremental paid claims and future case outstanding

The experiment code is publicly available:

<https://github.com/kasaai/deeptriangle>

The paper itself is available at:

<https://arxiv.org/pdf/1804.09253v4>

1. Where DeepTriangle comes from

It helps to begin with ordinary chain ladder.

Suppose $C(i, j)$ is cumulative paid claims for accident year i at development age j .

In simplified form, chain ladder assumes:

$$C(i, j+1) \approx f_j \times C(i, j)$$

where f_j is the development factor taking claims from development age j to development age $j+1$.

The important feature is that **the same development factor f_j is used across accident years**.

If the estimated development factor from year 3 to year 4 is 1.20, the model broadly applies that same 20% development relationship to all accident years currently at that maturity.

It is slightly misleading to say that accident year and development year are statistically independent.

A better description is:

> The development pattern is assumed to be homogeneous across accident years. There is no interaction allowing the development factor itself to change with accident year.

The familiar Generalized Linear Model, or GLM, representation makes this clearer.

A common over-dispersed Poisson formulation for incremental claims has:

$$\log E[C(i, j)] = c + \alpha_i + \beta_j$$

where:

- α_i is an accident-year effect;
- β_j is a development-year effect;
- c is an overall level.

What is absent is an interaction between accident year and development year.

The two effects are therefore separable.

This gives chain ladder much of its simplicity and stability. It can also

become restrictive when development behaviour itself changes over calendar time because of inflation, changes in claims handling, legal developments, changes in claim mix or other structural effects.

2. Wüthrich's step towards heterogeneous development

Mario Wüthrich's earlier neural-network work provides a useful bridge between traditional chain ladder and DeepTriangle.

Instead of assuming one development factor f_j at development age j , Wüthrich asks whether that factor can depend on additional claim characteristics x .

Instead of:

f_j

the model learns something closer to:

$f_j(x)$

The basic chain-ladder idea survives, but the factor can vary according to the characteristics of the underlying claim.

The progression is therefore:

Model	Main idea
Ordinary chain ladder	One development factor at each development age
Wüthrich	Development factor varies with claim characteristics
DeepTriangle	Learns the future development sequence directly

Wüthrich is essentially saying:

> Keep development factors, but allow them to become heterogeneous.

DeepTriangle goes further:

> Let the neural network learn the sequence dynamics directly rather than explicitly specifying the development-factor structure.

There is a trade-off.

Greater flexibility can improve predictive performance, but it does not make long-range extrapolation automatically reliable.

3. What is Mack chain ladder?

Kuo compares DeepTriangle with the **Mack chain-ladder model**.

Mack is not fundamentally a different way of calculating the central chain-

ladder reserve.

Its central estimates correspond to the familiar chain-ladder estimates.

What Thomas Mack adds is a ****stochastic framework****.

Ordinary chain ladder might tell the actuary:

> Best estimate reserve = £100 million.

Mack additionally asks:

> How uncertain is that £100 million?

Under assumptions about conditional expectations and variances, Mack allows the actuary to estimate quantities such as the standard error and mean squared error of prediction.

A useful shorthand is:

> ****Ordinary chain ladder gives the central estimate; Mack adds a statistical framework around that estimate.****

Its limitation is that it retains the underlying chain-ladder development structure.

If the development pattern itself becomes inappropriate, adding an uncertainty calculation around it does not repair the structural problem.

4. Bootstrap over-dispersed Poisson: a brief aside

Kuo also benchmarks against a ****bootstrap over-dispersed Poisson****, or bootstrap ODP, model.

The over-dispersed Poisson GLM models incremental claims using a structure such as:

$$\log m(i,j) = c + \alpha_i + \beta_j$$

and allows the variance to exceed the ordinary Poisson variance:

$$\text{Var}[C(i,j)] = \phi \times m(i,j)$$

The parameter ϕ is a dispersion parameter.

Under the standard reserving formulation, the resulting point estimates correspond closely to ordinary chain ladder.

The bootstrap then repeatedly resamples residual variation and refits the model. This produces many plausible reserve outcomes rather than one point estimate.

More developed bootstrap reserving procedures can also simulate future

process variation.

This is mainly an aside in the DeepTriangle story.

The useful contrast is simply:

> **DeepTriangle is evaluated primarily as a point forecaster, whereas bootstrap ODP has a natural route towards a predictive reserve distribution.**

5. The DeepTriangle dataset

Kuo uses **real insurance data**, not simulated claims.

The data come from National Association of Insurance Commissioners Schedule P triangles.

The four lines of business are:

- commercial auto;
- private passenger auto;
- workers' compensation;
- other liability.

There are **50 insurers per line of business**.

Kuo fits a separate DeepTriangle model for each line of business, while pooling information from the 50 insurers within that line.

So conceptually:

- commercial auto → one model using 50 insurers;
- private passenger auto → one model using 50 insurers;
- workers' compensation → one model using 50 insurers;
- other liability → one model using 50 insurers.

The observations remain aggregate triangle cells rather than individual claims.

6. The 1988–1997 dates: an important clarification

The phrase **1988–1997** can be misleading if it is read as the complete calendar span of the dataset.

Those are **accident years**.

They do not mean that all observations stop in calendar year 1997.

Each accident year ultimately has development experience through development year 10.

For example, the 1997 accident year continues developing after calendar

year 1997.

Its development year 10 would occur approximately in calendar year 2006.

This is crucial to understanding the experiment.

Kuo effectively asks:

> Suppose I am standing at 31 December 1997. What would I know at that date, and how accurately could DeepTriangle forecast everything that subsequently developed?

The observations becoming available after 1997 are hidden from the model during the forecasting exercise.

They nevertheless exist in the historical dataset.

They can therefore later be used as ****actual outcomes**** against which the forecasts are compared.

7. Paid, incurred and case outstanding

Kuo uses:

- cumulative paid loss;
- incurred loss;
- net earned premium;
- development lag;
- accident year;
- insurer code.

For this paper:

``case outstanding = incurred - cumulative paid``

Equivalently:

``incurred = cumulative paid + case outstanding``

So case outstanding is essentially the insurer's ****case reserve**** at that development date.

DeepTriangle converts cumulative paid losses into ****incremental paid losses****.

Both incremental paid and case outstanding are then divided by net earned premium.

An observation is therefore approximately:

``Y(i,j) = (incremental paid / premium, case outstanding / premium)``

This normalisation helps put insurers of very different sizes onto

comparable numerical scales during neural-network training.

8. Treating a triangle row as a sequence

This is the central conceptual departure from ordinary chain ladder.

Suppose an accident year currently has three development periods observed:

`Y1, Y2, Y3`

DeepTriangle does not simply estimate a factor taking `Y3` into `Y4`.

It uses the complete observed sequence:

`Y1, Y2, Y3`

to forecast:

`Y4, Y5, ..., Y10`

Each `Y` contains information about both:

- incremental paid claims;
- case outstanding.

This is a ****sequence-to-sequence forecasting problem****.

The simplest description is:

> sequence in → sequence out.

This is substantially more flexible than estimating one development factor at a time.

9. Why use a gated recurrent unit?

Kuo uses a ****gated recurrent unit****, or GRU.

A GRU is a type of recurrent neural network designed for sequential data.

A useful analogy is an actuary moving across an accident-year row while maintaining a running notebook.

At each development period the actuary asks:

> What from the earlier development history should I retain?

and:

> How much should the newest observation change my current view?

The GRU's hidden state performs something roughly analogous to this

notebook.

DeepTriangle uses an ****encoder-decoder architecture****.

The encoder reads the observed development history and converts it into an internal representation.

The decoder then converts that representation into future development outputs.

Both the encoder and decoder use 128 hidden units.

The actuarial consequence is important.

Two accident years might currently have the same cumulative paid amount but have reached it through very different development histories.

DeepTriangle can potentially distinguish between those histories.

A simple development factor may not.

10. Multi-task learning: why predict case outstanding?

DeepTriangle predicts two quantities simultaneously:

1. future incremental paid claims;
2. future case outstanding.

This is called ****multi-task learning****.

The idea is that case outstanding contains information about the claims process that may also help predict future payments.

It provides the network with another training signal and may encourage it to learn representations of claims development that generalise better than those obtained from paid claims alone.

There is an important actuarial caveat.

Case reserves reflect more than underlying claim severity.

They can also reflect:

- claims-handler judgement;
- reserving philosophy;
- settlement strategy;
- operational procedures;
- changes in case-reserving guidelines.

If a company materially changes its case-reserving practices, the historical relationship between case estimates and future payments may change.

Case estimates are therefore informative partly because they incorporate human judgement.

That makes them valuable, but it also creates another possible source of model instability.

11. Company embeddings

The insurer identifier is categorical.

Giving the network company numbers such as 1, 2 and 50 directly would incorrectly suggest that they have numerical meaning.

DeepTriangle instead learns an ****embedding**** for each insurer.

Each of the 50 companies is represented by a learned 49-dimensional vector.

Kuo suggests that these vectors may indirectly represent persistent characteristics such as:

- company size;
- case-reserving philosophy;
- persistent development behaviour.

Companies that behave similarly for the forecasting problem should acquire similar representations.

It is tempting to describe this as:

> Fit the claims history first, then let the company embedding explain the remaining residual.

That is not literally what happens.

The embedding and the rest of the neural network are trained ****simultaneously****.

There is no separate residual-fitting stage.

A better actuarial analogy is a very flexible ****company-specific effect****.

The sequence explains whatever development behaviour it can, while company identity gives the network another way to capture persistent insurer-specific differences.

The caveat is that 49 dimensions for only 50 insurers is extremely flexible.

The embedding might partly learn insurer identity rather than a compact set of genuinely transferable characteristics.

A completely new company also has no learned embedding. Kuo notes that the embedding input has to be enlarged and the model refitted when new

companies are introduced.

12. Direct multi-step forecasting

This is one of the most interesting features of DeepTriangle.

DeepTriangle is ****not**** a conventional recursive one-step forecasting model.

A recursive one-step model might work as follows:

``Y1, Y2, Y3 → predicted Y4``

It then appends that prediction:

``Y1, Y2, Y3, predicted Y4 → predicted Y5``

It then repeats the process again to obtain ``Y6``, and so on.

This creates a familiar problem.

If the model makes an error when predicting ``Y4``, that incorrect prediction becomes part of the input used to predict ``Y5``.

Errors can therefore propagate through the forecast horizon.

DeepTriangle instead uses ****direct multi-step forecasting****.

There are ten development periods.

At least the first development period is observed, so the maximum future forecast horizon is ****nine periods****.

For the newest accident year the model can conceptually do:

``Y1 → predicted Y2, Y3, Y4, ..., Y10``

The decoder therefore has a maximum of ****nine future output positions****.

Importantly, the predicted value at one horizon is ****not fed back into the entire model as the input used to obtain the next horizon****.

This avoids the classic recursive forecasting feedback loop.

This also explains one difference between DeepTriangle and the automated machine-learning benchmark in Kuo's experiment.

The conventional machine-learning models produce scalar outputs.

They therefore have to forecast iteratively.

DeepTriangle was specifically designed to produce the future development sequence directly.

13. Different accident years need different forecast horizons

Standing at 31 December 1997, different accident years have reached very different levels of maturity.

Accident year	Observed by 1997	Future periods
---	---	---
1989	9	1
1990	8	2
1994	4	6
1996	2	8
1997	1	9

The 1989 accident year therefore requires only one additional forecast.

The 1997 accident year requires nine.

DeepTriangle nevertheless uses one fixed neural-network architecture.

14. Padding and masking

DeepTriangle handles different sequence lengths using ****padding and masking****.

Suppose a particular input history contains only:

```
`Y1, Y2, Y3`
```

but the neural network expects a sequence with nine positions.

The unused positions can be padded with a special placeholder value.

Kuo's code uses ``-99``.

Conceptually the input might look like:

```
`Y1, Y2, Y3, -99, -99, -99, -99, -99, -99`
```

A ****masking layer**** tells the neural network:

> These ``-99`` positions are padding. Do not treat them as genuine claims observations.

A similar approach is used for the target sequence.

Suppose only three genuine future observations are available for a particular training example.

Its target can effectively look like:

```
`Y4, Y5, Y6, MASK, MASK, MASK, MASK, MASK, MASK`
```

The model still has a fixed nine-position output structure.

However, a ****masked loss function**** excludes the padded positions from the training loss.

So the network is not penalised for predictions corresponding to development periods for which no legitimate target exists.

This is an elegant implementation detail.

It allows one neural-network architecture to learn from:

- very immature accident years;
- moderately developed accident years;
- almost fully developed accident years.

At inference, outputs corresponding to development periods beyond development year 10 are simply not required.

15. What happens for 1989 versus 1997?

This makes the architecture particularly easy to understand.

For accident year 1989, at year-end 1997 the model already has nine development periods.

It therefore needs only:

`predicted development year 10`

The decoder architecture can still produce its standard sequence of outputs, but only the relevant first future position is required.

For accident year 1997, only development year 1 is known.

The model therefore needs:

`predicted development years 2, 3, 4, ..., 10`

All nine future positions are relevant.

So the same model architecture handles both extremes.

16. The 100 neural networks

For each line of business, Kuo trains an ensemble of ****100 separate neural networks****.

They have:

- the same architecture;

- the same modelling objective;
- the same general training data;

but different random initial neural-network weights.

So there really are:

`Model 1, Model 2, ..., Model 100`

All 100 are trained.

At inference, the 100 predictions are averaged.

In simple notation:

`ensemble prediction = average(prediction 1, ..., prediction 100)`

The reason is that neural-network optimisation is stochastic.

Two networks with the same architecture and data but different random starting weights can converge to somewhat different fitted functions.

Averaging many networks reduces some of this optimisation variability and generally produces a more stable central forecast.

With four lines of business and 100 models per line, Kuo therefore fits approximately:

`4 × 100 = 400 neural networks`

There are two distinct stages:

****Training****

Train 100 independently initialised models.

****Inference****

Run the fitted models and average their forecasts.

The ensemble is therefore principally a ****variance-reduction technique for the point prediction****.

17. The 100 models are not a reserve distribution

Kuo explicitly says that the variation among the 100 fitted networks should not be interpreted as a reserve predictive distribution.

If all 100 models give similar forecasts, that tells us that the result is relatively insensitive to random weight initialisation.

If the forecasts differ substantially, that tells us that the neural-network optimisation is unstable.

That is useful information.

But it does not capture all the uncertainty in future claim payments.

In particular, it does not provide a full measure of future **process uncertainty**.

18. Predictive uncertainty: a possible extension

DeepTriangle itself does not develop a full predictive uncertainty model.

One natural neural-network extension would be **quantile regression using pinball quantile loss**.

Instead of training the network only towards the conditional mean, different output heads could directly learn selected conditional quantiles.

For example:

- 2.5th percentile;
- 50th percentile;
- 97.5th percentile.

Conceptually:

`claim history → lower quantile, median, upper quantile`

This would be considerably more meaningful for predictive intervals than interpreting the variation caused by Kuo's random initialisations as a reserve distribution.

Ronald Richman discusses this approach in:

Mind the Gap: Safely Incorporating Deep Learning Models into the Actuarial Toolkit

He demonstrates how neural networks can use **pinball loss** to estimate predictive quantiles directly.

This is an interesting possible extension to DeepTriangle, but it is not something Kuo implements in the paper.

19. Training, validation and test data

Three different concepts need to be kept separate.

Training information

Kuo states that data **available as of year-end 1997** are used for fitting the model.

Think of the actuary standing at:

****31 December 1997****

and asking:

> What information would genuinely have been known at this date?

Validation

Within the fitting data, Kuo uses later observations becoming available after calendar year 1995 as a validation subset for ****early stopping****.

Each network can train for up to 1,000 epochs.

If validation performance fails to improve for 200 epochs, training stops and the model returns to the weights from the best validation point.

So 1995 is mainly relevant to the ****training/validation split****.

It is not the final forecasting cut-off.

Test data

Actual development occurring ****after calendar year 1997**** is hidden from the model during fitting.

Those later observations are subsequently used as holdout actuals.

That is what makes the final evaluation genuinely out-of-time.

20. The 1997 accident year really is projected almost all the way

This is worth making explicit because the terminology is confusing.

The dataset contains accident years:

****1988 to 1997****

but their subsequent development continues beyond 1997.

Standing at year-end 1997:

Accident year	Known	Future
1988	Dev 1-10	None
1989	Dev 1-9	Dev 10
1990	Dev 1-8	Dev 9-10
...	...	...
1996	Dev 1-2	Dev 3-10
1997	Dev 1	Dev 2-10

So for the 1997 accident year, DeepTriangle is effectively making a ****nine-**

development-period forecast**.

The model can later be assessed because the historical dataset contains the actual subsequent experience.

Those actual observations are deliberately withheld during the simulated year-end-1997 forecasting exercise.

For accident year 1997, development year 10 would occur at approximately calendar year 2006.

So this is substantially more demanding than a simple:

```
> train through 1995 and test during 1996-1997
```

experiment.

21. Forecast horizon matters

The newest accident years are the ****least mature**** and therefore require the longest forecasts.

For 1989, DeepTriangle needs essentially one future period.

For 1997, it needs nine.

Longer forecast horizons provide more opportunity for:

- model misspecification;
- inflation changes;
- claims-handling changes;
- unusual claim emergence;
- legal changes;
- changes in portfolio mix;
- general divergence from historical development patterns.

One particularly useful analysis would therefore have been prediction error by forecast horizon.

For example:

- one-step forecast error;
- three-step forecast error;
- six-step forecast error;
- nine-step forecast error.

Kuo's headline measures instead focus mainly on ultimate-loss accuracy across companies.

22. Delayed company data

The paper does not directly test operational reporting delays.

However, the sequence architecture has some natural tolerance for different observed history lengths.

Suppose an accident year should currently have:

`Y1, Y2, Y3, Y4`

but the most recent data have not arrived.

The model only has:

`Y1, Y2, Y3`

DeepTriangle can still forecast.

However, it must now begin forecasting one development period earlier.

Instead of predicting:

`Y5 to Y10`

it effectively has to predict:

`Y4 to Y10`

A reporting delay therefore ****lengthens the forecast horizon****.

This matters more for immature accident years.

Losing one observation when nine periods are already known is usually less serious than losing one observation when only two periods existed.

23. Inconsistent valuation dates

Pooling many companies creates another operational issue.

Suppose:

- Company A has reported through December;
- Company B has reported only through September;
- Company C has revised its June triangle;
- Company D has not yet submitted its latest data.

There is no longer one clean common valuation date.

A production implementation would therefore need strict ****as-of-date controls****.

The principle should be:

> Only information genuinely available by the valuation date may be used in the model.

Otherwise information from the future can leak into historical training or forecasting examples.

That would create look-ahead bias and artificially strong back-testing performance.

24. Would an insurer really have data from 50 competitors?

Probably not at the frequency needed for a live reserving process.

Schedule P is useful for research because it provides comparable regulatory information across many insurers.

A company would generally not have competitors' current internal payment and case-reserve triangles quickly enough to reproduce Kuo's setup operationally.

Kuo therefore suggests a potentially more realistic use inside an insurance group.

For example, the same architecture could share information across:

- legal entities;
- subsidiaries;
- geographical portfolios;
- product portfolios.

The company embedding could then represent persistent differences among those related portfolios.

25. Evaluation

Kuo compares DeepTriangle with:

- Mack chain ladder;
- bootstrap over-dispersed Poisson;
- Bayesian reserving models;
- an automated machine-learning benchmark.

The automated machine-learning benchmark includes methods such as:

- random forests;
- extremely randomised forests;
- gradient boosting;
- feed-forward neural networks;
- stacked ensembles.

Performance is assessed mainly using **Mean Absolute Percentage Error**, or MAPE, and **Root Mean Squared Percentage Error**, or RMSPE.

MAPE is approximately:

```
`average(|prediction - actual| / actual)`
```

RMSPE is approximately:

```
`square root of average(((prediction - actual) / actual)^2)`
```

RMSPE penalises large errors more heavily because the percentage errors are squared.

26. Results

DeepTriangle obtains the lowest reported MAPE and RMSPE across all four lines of business in Kuo's experiment.

Line	Mack MAPE	DeepTriangle
---	---:	---:
Commercial auto	6.0%	**4.3%**
Other liability	13.4%	**10.9%**
Private passenger auto	3.8%	**2.5%**
Workers' compensation	5.3%	**4.6%**

For RMSPE:

Line	Mack RMSPE	DeepTriangle
---	---:	---:
Commercial auto	8.0%	**5.7%**
Other liability	20.2%	**15.0%**
Private passenger auto	6.1%	**3.9%**
Workers' compensation	7.9%	**6.7%**

The results are encouraging, particularly for private passenger auto and commercial auto.

But the appropriate conclusion is narrow:

> **On this particular Schedule P experiment, with these four lines of business, this model architecture and these evaluation measures, DeepTriangle produced better ultimate-loss point forecasts than the tested benchmarks.**

That is not the same as saying:

> Deep learning universally beats chain ladder.

27. An important failure case

Kuo also looks at individual company results.

DeepTriangle captures the development of Company 1767's commercial-auto business reasonably well.

However, it fails to anticipate deteriorating workers' compensation development for Company 337.

This is one of the most useful findings in the paper.

A recurrent neural network can learn complicated historical patterns.

It cannot reliably forecast a genuine regime change when the preceding data contain insufficient warning.

Possible real-world causes might include:

- sudden claims inflation;
- court decisions;
- claims-handling deterioration;
- legislative changes;
- changing settlement behaviour;
- changing claim mix.

More sophisticated pattern recognition does not make genuinely unforeseen changes predictable.

28. What DeepTriangle really contributes

The most important contribution is not simply:

> a GRU applied to a triangle.

The progression is better understood as follows.

****Chain ladder****

`current maturity + development factor → future claims`

****Wüthrich****

`claim information + claim characteristics → heterogeneous development factor`

****DeepTriangle****

`whole development sequence + case-reserve behaviour + insurer identity → future development sequence`

DeepTriangle therefore reframes aggregate reserving as:

> ****supervised multivariate sequence forecasting****

Paid and outstanding development are learned jointly.

Information is shared across insurers.

The complete observed history can influence the forecast.

Company-specific behaviour can be represented through embeddings.

And future development is forecast ****directly across multiple horizons rather than recursively feeding predictions back into the model****.

That last feature is particularly important.

29. What DeepTriangle does not solve

Several important reserving problems remain.

****Predictive uncertainty****

DeepTriangle primarily produces point forecasts. Quantile regression using pinball loss would be one possible extension.

****Structural breaks****

Historical sequence learning cannot guarantee performance when the underlying claims process changes.

****Explainability****

A chain-ladder development factor can be inspected directly. A GRU combined with company embeddings is much harder to explain.

****New companies****

A previously unseen insurer does not have a learned embedding and requires model refitting.

****Operational data availability****

The experiment benefits from harmonised cross-company regulatory information that may not be available quickly enough in practice.

****Long-range extrapolation****

The newest accident years contain the least observed information and require the longest forecasts.

30. Overall assessment

DeepTriangle is interesting because it does ****not**** abandon the actuarial development triangle.

Instead, it treats the triangle as a richer machine-learning object.

The accident-year row becomes a sequence.

Paid claims and case outstanding become related prediction tasks.

Company identity becomes a learned representation.

Information is shared across many insurers.

And rather than explicitly imposing development factors, the model learns a nonlinear mapping between the observed development history and future development.

The model therefore occupies an interesting middle ground between traditional aggregate reserving and individual-claims reserving.

Its strongest result is ****point prediction****.

Its weakest area is ****uncertainty quantification****.

Mack chain ladder and bootstrap ODP remain much more naturally connected to conventional reserve-risk analysis.

A neural quantile approach using ****pinball loss****, as demonstrated by Ronald Richman, would be one plausible way to extend DeepTriangle towards predictive intervals.

The Company 337 failure is also an important reminder:

> ****More sophisticated pattern recognition does not make genuinely unforeseen development predictable.****

Five-sentence recap

DeepTriangle is a ****supervised aggregate reserving model**** using real Schedule P development triangles rather than individual claims.

It uses a gated recurrent unit encoder-decoder to forecast incremental paid claims and case outstanding jointly, while company embeddings capture persistent insurer-specific behaviour.

An important feature is that DeepTriangle is a ****direct multi-step forecaster****: its decoder can produce up to nine future development outputs, while padding and masking allow the same model architecture to handle shorter observed and forecast sequences.

The accident years are 1988–1997, but the underlying claims experience continues beyond calendar year 1997, allowing the 1997 accident year to be forecast from development year 1 through development year 10 and subsequently compared against the actual development that emerged.

Kuo's 100-model ensembles stabilise the point forecasts rather than providing a reserve distribution; if predictive intervals were required, an extension based on ****pinball quantile loss**** would be a more natural neural-network approach.

Concepts the reader should now be able to explain

- Why chain ladder assumes broadly homogeneous development across accident years.
- Why the GLM representation has separable accident-year and development-year effects.
- How Wüthrich allows development factors to depend on claim characteristics.
- What Mack adds to ordinary chain ladder.
- What bootstrap ODP contributes and why it is only an aside here.
- Why DeepTriangle remains an aggregate reserving method.
- What a gated recurrent unit contributes to triangle modelling.
- Why paid claims and case outstanding are predicted jointly.
- How company embeddings behave roughly like flexible insurer-specific effects.
- Why DeepTriangle is a ****direct multi-step**** rather than recursive one-step forecaster.
- Why its maximum forecast horizon is nine development periods.
- How padding and masking allow one fixed architecture to work with different triangle maturities.
- Why 1988–1997 refers to ****accident years****, not the complete calendar span of the underlying claims experience.
- How subsequent actual development allows the forecasts for recent accident years to be genuinely back-tested.
- Why Kuo trains 100 networks per line of business.
- Why those 100 networks do not themselves constitute a reserve distribution.
- How pinball quantile loss could extend a neural reserving model towards predictive intervals.
- Why delayed triangle submissions effectively increase the required forecast horizon.
- Why long-horizon forecasts for the newest accident years are the hardest part of the exercise.

Main references

Kevin Kuo, ****DeepTriangle: A Deep Learning Approach to Loss Reserving****

<https://arxiv.org/pdf/1804.09253v4>

DeepTriangle code:

<https://github.com/kasaai/deeptriangle>

Ronald Richman, ****Mind the Gap: Safely Incorporating Deep Learning Models into the Actuarial Toolkit****

<https://www.cambridge.org/core/journals/british-actuarial-journal/article/mind-the-gap-safely-incorporating-deep-learning-models-into-the-actuarial-toolkit/4716B70DF7B06258DBB42F068790B156>